Orientation: Boundary Integrals in `lappy`

A map of the machinery, and where the difficulty lives

lappy development notes

2026-08-10

Abstract

A map, not a lab notebook. `benchmarks/suite/run/NOTEBOOK.md` (2700+ lines) records *how* things were found; this records *what is true now* and *where the difficulty lives*. Read this first; go there for evidence.

The companion documents are `corner_quadrature.pdf` (the research record, which derives the corner rule properly) and `boundary_quadrature_primer.pdf` (a gentler introduction for someone new to the project).

1 Why boundary integrals at all

Two formulas, one instrument:

$$\text{Rellich} \qquad \int_\Omega u^2 \,\mathrm{d}x = \frac{1}{2\lambda} \oint_{\partial\Omega} (r \cdot N) \Big(\frac{\partial u}{\partial n}\Big)^2 \,\mathrm{d}s, \qquad r = x - x_0, \tag{1}$$

$$\text{Hadamard} \qquad d\lambda = -\oint_{\partial\Omega} \Big(\frac{\partial u}{\partial n}\Big)^2 (V \cdot n) \,\mathrm{d}s, \qquad V = \text{boundary velocity}. \tag{2}$$

Both turn a statement about the *domain* into an integral over the *boundary*, against the same Cauchy data $\partial u/\partial n$, on the same nodes, with the same weights. Only the weight function differs: $r \cdot N$ for the norm, $V \cdot n$ for the shape derivative.

This matters because MPS produces $\partial u/\partial n$ on the boundary natively. So **normalizing an eigenfunction** (needed for anything quantitative) and **differentiating an eigenvalue with respect to shape** (needed downstream, for optimization and inverse problems) are the *same computation with a different weight*. One quadrature investment buys both. The alternative — interior cubature for the norm — needs volume points and has measured as the fragile half every time the two were compared.

The case $V = x - x_0$ (uniform dilation) makes (2) reduce to (1) exactly, which pins the sign and scale of the shape derivative for free. `gram()` and the shape derivative are literally the same call to `weighted_integral(ed, 'NN', ·)` with different weights.

Consequence. Essentially all of `lappy`'s accuracy risk is concentrated in *one boundary quadrature*. That is a deliberate bet, and it is working. It also means a defect there is systemic, which is why so much effort has gone into it.

2 Where the difficulty actually lives

At a corner of interior angle α, with $\nu = \pi/\alpha$, a Dirichlet eigenfunction expands as $u = \sum_k c_k J_{k\nu}(\sqrt{\lambda}\, r)\sin(k\nu\theta)$, so on an edge leaving the corner

$$\frac{\partial u}{\partial n} \sim r^{\nu-1}F(r), \qquad \Big(\frac{\partial u}{\partial n}\Big)^2 \sim r^{2\nu-2}G(r).$$

The integrand carries a **non-integer power of arclength** at every corner. Gauss–Legendre converges only algebraically on τ^γ (as $n^{-(2\gamma+2)}$), not spectrally, so an unadapted rule loses most of the precision. Everything below is machinery for that one fact.

Note carefully: this is **not** only about reentrant corners. The power $r^{2\nu-2}$ is non-smooth whenever $2\nu - 2$ is not a non-negative even integer, which includes plenty of *convex* corners ($\nu = 1.4$ gives $r^{0.8}$). That is why the regular n-gons need corner rules at all.

3 The case table

This is the thing to keep in your head. `boundary_quadrature()` classifies every corner, then every panel.

3.1 Which rule a corner gets

The test for "does this corner need special treatment" is **measured, not assumed**. The function `quad.smooth_power_error` is asked directly whether plain Gauss–Legendre can integrate $\tau^{2\nu-2}$ to the target; if it cannot, the corner gets an adapted rule. (Switch `nonintegral`, on by default.)

corner class	edges	rule	sub	why
smooth rule suffices	either	Legendre	—	plain Gauss already reaches the target; nothing to fix
$2/\nu$ an integer	**straight**	`cornerjac`	ν	family is the sparse $\{j\nu + 2q\}$, which $t = r^\nu$ rationalizes. Among reentrant angles this is **only** $\alpha = 3\pi/2$
everything else, including irrational ν	curved, or straight with $2/\nu \notin \mathbb{Z}$	`cornerinterp`	ν	family is the dense $\{j\nu + m\}$; no substitution rationalizes it without crushing nodes, so exactness comes from the *weights* while the nodes keep the mild sub $= \nu$ placement

Three further classes get no corner rule at all, because none would help:

demoted class	why, and what is reported
$\nu \le 1/2 + \varepsilon$ (near-slit)	the Rellich integrand is barely integrable and no rule recovers it. Precision is reported as ∞. Remedy: place x_0 on that corner (§4)
mixed BC (Zaremba corner)	the integrand $\sim r^{\nu-2}$ is not integrable for $\nu < 1$, so the identity itself fails here
non-Dirichlet `bc_type`	only Dirichlet corners are wired; a Neumann corner needs different exponents on the `uv` and `TT` kernels

Demotions are never silent: `shortfalls` names the corner and the reason, `bq.precision` becomes ∞, and a warning is raised. `singular_corner_report(domain)` prints the whole classification and is the first thing to look at when accuracy disappoints.

3.2 Which weight you are integrating

The corner rule is built for the eigenfunction's own exponent family. A *weight* multiplies that, and whether the product stays in the exact class depends on the weight's behaviour at the corner.

weight	corner behaviour	in class?	what to pass
$r \cdot N$, straight edge	exactly **constant** ($m = 0$)	yes	default `weight_family='even'`
$r \cdot N$, curved edge	analytic series in s, integer powers	yes	default
$V \cdot n$ vanishing to even order	even integer powers	yes	default
$V \cdot n$ **moving a corner** ($\sim r$)	**odd** integer powers	**no**	`weight_family='integer'`

The last row is the one that bites, and it is exactly what a shape-optimization parametrization supplies. With the default sub $= \nu$, a weight r^m becomes $t^{m/\nu}$ — a non-integer power with a *small* exponent, so Gauss decays only as $n^{-(2m/\nu+2)}$. Measured on a 1.5π sector against 40-digit truth:

	$p = 0$	$p = 1$	$p = 2$	$p = 3$
sub $= \nu$, order 32	2.9e-14	**4.6e-07**	8.5e-14	4.0e-14
sub $= 1/2$, order 16	4.7e-15	**1.2e-14**	1.1e-14	1.0e-14

`weight_family='integer'` switches singular corners to sub $= 1/2$, which makes every integer power the exact polynomial t^{2m} while sending the Bessel family to $t^{2j\nu}$ — non-integer, but with exponents growing by 2ν per term, which Gauss resolves at once. It assumes *nothing* about ν, so it covers the generic arc–arc corner where no exact substitution exists. End-to-end on $d\lambda/d\alpha$ this is 6–8 orders, at fewer nodes in half the cases.

The default stays `'even'`: it is equally accurate and cheaper for everything `lappy` integrates itself, and changing it would perturb every recorded reference value.

3.3 The smooth part of the boundary

Away from corners, panels use plain Gauss–Legendre, with two independent sizing questions.

question	function	notes
resolve the *oscillation* $e^{ik\tau}$	`smooth_order_for_precision`	$k = \sqrt{\lambda_{\max}} \cdot$ (panel arclength)
resolve the *geometry*	`geometry_order_for_precision`	curved segments only; asks whether the arclength reparametrization itself is resolved. `resolve_geometry=True` by default

4 The methods, one at a time

`cornerjac`. Gauss–Jacobi after the substitution $t = r^{\text{sub}}$. Exact when the substitution maps the corner's exponent family to integers. Cheap (order 8 suffices at $\alpha = 3\pi/2$ where `cornerinterp` needs 24). Its limitation is node placement: $\tau = t^{1/\text{sub}}$ crushes the innermost node as sub shrinks, and once $\tau_{\min}$ falls below `_KRESS_TAU_FLOOR` $= 10^{-9}$ the node rounds onto the corner itself under a segment's parametrization, where a basis's $1/r$ terms are fatal. That is why sub $= 1/q$ for $\nu = p/q$ — which *would* rationalize the dense family exactly — is not used for placement: $q \geq 4$ puts $\tau_{\min}$ below the floor.

`cornerinterp`. Nodes from `cornerjac`'s mild sub $= \nu$ placement, weights solved so the rule is exact on the corner's actual exponent set. It gets exactness from the weights instead of the coordinates, so $\tau_{\min}$ stays at $\sim 10^{-5}$. The weight solve is deliberately *under-determined* (`n_exp < order`, minimum-norm least squares), which keeps $\sum |w| = 1$; the square solve is exact on the span but has $\text{cond}(V) \sim 10^{19}$ and weights reaching -10^4.

Known limit. `cornerinterp` cannot be pushed further by raising `n_exp` — $\text{cond}(V)$ goes $8\times10^6 \to 5.7\times10^{15} \to 4.4\times10^{18}$ — and it is *not* an arithmetic problem: rebuilding the same rule at 60 dps makes it worse, with $\sum |w|$ exploding to 4×10^{10}. The Jacobi nodes are simply the wrong nodes for the dense family. Fixing that properly means solving for nodes *and* weights jointly (a true generalized Gaussian rule, Ma–Rokhlin). Not attempted, and sub $= 1/2$ (§3.2) made it unnecessary for the case that motivated it.

Panel structure. Corner panels are anchored at the corner and cover a fraction `panel_frac` of the segment; that fraction is halved when both ends of a segment are singular. A *clearance cap* limits the panel to a fraction of the distance to the nearest non-adjacent boundary piece, because the corner expansion stops being valid beyond it.

The reference point x_0. Identity (1) holds for *every* x_0, which makes x_0-invariance a free diagnostic: any variation across x_0 is pure quadrature error. `default_x0` puts it at the most singular corner, where $r \cdot N$ vanishes identically on both edges — removing that corner from the integrand rather than resolving it. Free, but it can only ever cover one corner.

5 The two ways to know it worked — and the open decision

	mechanism	what it is
a priori	`corner_model_error` → `bq.precision`	*predicts* a rule's error from a model of the integrand class, before computing anything
a posteriori	`refine_quadrature` → `verify_gram`	*measures*, by recomputing on a refined rule and reporting what moved

This is the live architectural question. `bq.precision` is currently treated as an advertised precision, but it is a single scalar standing in for a whole class of integrands, and it is measurably not up to that job.

- Calibrated against closed-form truth over 3822 configurations (`benchmarks/eigfun_quad/corner_model_calibration.py`), worst-case **optimism is nine orders** — it promises precision it does not deliver. It is also pessimistic by up to 3×10^5 at convex ν, which inflates node counts. Two causes pull in opposite directions: unsigned Bessel coefficients (pessimistic) and a fixed series depth that cuts at the peak once $k > 8$ (optimistic). Since k reaches 10.6 across the suite, both are live.
- Three candidate replacements were built and **all three rejected on measurement** — each fixed one regime and broke another. The instrument landed; the fix did not.
- More fundamentally: `chevron_1_2` claims 10^{-13} while `verify_gram` measures 4.9×10^{-8}, and that gap is *identical* under every corner model tried. The error is not at the corners — it comes from the smooth panels, whose requirement is set by the **basis's own singularity structure** (where the fundamental solutions put their poles), which no geometry-only model can ever see.

So a perfect corner model still would not deliver the advertised number. The open proposal is to **demote `bq.precision` to a sizing heuristic and let `verify_gram` be what certifies**. The corner model would then only need to land in the right ballpark, and its calibration would leave the critical path. Not yet decided.

6 State of play

Settled and load-bearing.

- Certification runs on boundary norms (ε is scale-invariant, so this is both sound and faster).
- `FundamentalBasis` rejects sources inside the domain — those columns are not particular solutions, which voids the MPS premise *and* the Moler–Payne hypothesis. This was a real correctness bug; its signature is a suspiciously *low* tension background.
- `weight_family='integer'` for corner-moving shape velocities (§3.2).
- `tests/test_shape_derivative.py` — the Hadamard contract, 12 tests, ∼2 s.
- Benchmark tally 39 / 3 / 2 (8+ digits / complete but fewer / failure).

Open.

- Corner order model calibration — instrumented, unsolved (§5).
- Claimed versus measured precision — the architectural decision above.
- Basis-selection heuristics: `make_default_basis(domain, lam_max, precision)` is sketched in `docs/basis_heuristics.md`, not built.
- Buckets 2 and 3: `stadium`, `mushroom_neck01`, and the two spirals.
- Still unbuilt from the Hadamard plan: the Tier-3 finite-difference benchmark and the objective study (λ digits versus $d\lambda$ digits across bases).

7 Where to read further

topic	**file**
the identity and its derivation	`docs/rellich.md`, `rellich_hadamard_mps.pdf`
corner quadrature theory (research record)	`docs/corner_quadrature.pdf`
gentler introduction, for someone new	`docs/boundary_quadrature_primer.pdf`
the integration API	`docs/eigfun_integrals.md`, `boundary_integrals.md`
what lappy owns vs. a downstream shape package	`docs/scope_and_downstream.md`
evidence, chronologically	`benchmarks/suite/run/NOTEBOOK.md`, `FINDINGS.md`
per-domain status	`benchmarks/suite/run/BUCKETS.md`